NoCap: Fact-Checking with AI User/Developer Manual

Team Members:

Thomas Chamberlain tchamberlain2023@my.fit.edu
Anthony Ciero aciero2022@my.fit.edu
Varun Doddapaneni vdoddapaneni2023@my.fit.edu
Josh Pechan jpechan2023@my.fit.edu

Client and Faculty Advisor:

Dr. Marius Silaghi

TABLE OF CONTENTS

1. Introduction
2. User Manual
    2.1 Navigation/Sidebar
    2.2 Home Page
        2.2.1 Generating an Article Authenticity Report
        2.2.2 Example Articles
    2.3 Report Page
        2.3.1 Article Metadata
        2.3.2 Authenticity Report Information
        2.3.3 Like/Dislike Feature
        2.3.4 Graphical Analysis
    2.4 Database Page
        2.4.1 Report Cards
        2.4.2 Publisher Cards
        2.4.3 Searching/Filtering Database
    2.5 Chrome Extension
3. Developer Manual
    3.1 Frontend
        3.1.1 Pages
            3.1.1.1 App
            3.1.1.2 Layout
            3.1.1.3 HomePage
            3.1.1.4 ReportPage
            3.1.1.5 DatabasePage
        3.1.2 Components
            3.1.2.1 Sidebar
            3.1.2.2 InputBar
            3.1.2.3 SearchBar
            3.1.2.4 FilterMenu
            3.1.2.5 ReportCard
            3.1.2.6 PublisherCard
            3.1.2.7 AnalysisCharts
        3.1.3 AppRoutes

3.2 Backend

3.2.1 Database

3.2.2 AWS Amplify

3.2.3 AWS AI Nova

3.2.4 API

3.2.5 AI Prompt

3.3 System Architecture

1. Introduction

The purpose of this document is to describe the functionality and important information about project NoCap to both users and developers alike. The goal of the project is to allow users to fact check articles using the ever growing knowledge and data of AI. Users can create authenticity reports that will summarize and score a given article, which is then uploaded to our database where all users can go and see articles that have been fact checked so this information is widely available.

2. User Manual

The user manual will teach the user about the different parts of the website and what features can be used and tasks can be performed with each aspect of the website. This includes how to navigate throughout the website, create an authenticity report, view the authenticity report and what information it contains, search the database of authenticity reports, filter options to view reports based on certain criteria, and select articles in the database to view their reports.

2.1 Navigation/Sidebar

On all pages of the website, the navigation bar is shown on the left side. This consists of a few features. The top button is the hamburger, which simply expands the sidebar to show the description of each button. When open this button turns into an X which will reclose it. The next three buttons are used for navigation throughout the pages of the website. The first being the Home Page button, which leads the user to the home page. The home page is the default page when opening the website, but is used to return to the home page after further navigation through the website. The second is the Report Page button, which brings the user to the last authenticity report that was generated. If no authenticity reports have been generated previously, this button does nothing. The last navigation button is the Database page button, which brings the user to the authenticity report database. Here users can view all reports generated by all website users. The final button on the sidebar is used for switching the website between light and dark modes so users can choose their preference when viewing the website. Lastly, the NoCap logo can be seen at the bottom of the navigation bar.

2.2 Home Page

The Home Page is the default page when opening the website. This page consists of the NoCap name and a prompt to “Paste an article URL below to check its factual accuracy.” Its main feature is the input bar where the user will input the URL of the article they want to fact check and the submit button. Below is Example Picks, which pulls three articles from the database, one with the highest authenticity score, one in the middle, and the lowest.

2.2.1 Generating an Article Authenticity Report

The main way for a user to generate an article authenticity report is to paste an article URL into the input bar. Once the URL is pasted in, the user can either click the submit button or press the Enter key while the input bar is still selected. Doing this will create an authenticity report for viewing that appears in the Report Page.

2.2.2 Example Articles

The other features of the Home Page is the Example Picks. This section shows three example articles that are pulled from the database. These three articles consist of an article in the database with the highest authenticity score, one with a score in the middle, and one with the lowest score. Clicking on these articles will bring the user to the Report Page and show the authenticity report of the article.

2.3 Report Page

The report page is the main page for article authentication and viewing the report. After the user generates or pulls up an existing report from either the Database Page or the Example Picks on the Home Page, the Report Page opens showing all the information of the article authenticity report.

2.3.1 Article Metadata

The first part is the article meta data; this includes the article title at the top, the article's author, publisher, and publication date, and the link to the article for reading.

2.3.2 Authenticity Report Information

The second part consists of the report information; this includes the authenticity score from zero to one hundred; green when the score is >=75, yellow when it's >=50, and red when it's any lower, the summary which is a quick description of why the article received the score it did, and the detailed analysis, which is a paragraph about the specific parts of the article, why they are fact or fiction, and the justification for the article score.

2.3.3 Like/Dislike Feature

The next aspect of the Report Page is the Like/Dislike feature. This allows users to like or dislike the authenticity report depending on if it was helpful or not in determining the authenticity of the article.

2.3.4 Graphical Analysis

The final part is the graphical analysis. This shows the user 2 different graphical features. The first is an Articles Claims bar chart, showing a visualization of the amount of correct and incorrect claims made in the article. The second is a Descriptor Sentiment horizontal bar chart, showing

the number of positive, neutral, and negative descriptors used before people and entities, labeled by colors green, yellow, and red respectively. The y-axis is the specific descriptors and the x-axis is the frequency of these descriptors.

2.4 Database Page

The Database Page is where all the user created article authenticity reports are stored so that any user can search through the database to see reports for articles that may be already fact checked. If the user tries to create an article report for one that is already stored in the database, it will just fetch the report information.

2.4.1 Report Cards

The main feature of the Database Page are the article report cards. These cards are small sections in the database that each represent an article and its corresponding authenticity report. The card contains the article metadata, including the article's title, author, publisher, publication date, and URL. It also contains the authenticity score in its corresponding color, as well as shows the amount of likes and dislikes the report has. The user can also like and dislike the report from the report card in the Database Page.

2.4.2 Publisher Cards

The article report cards are stored by default in publisher cards. These publisher cards simply hold any article that is stored in the database that is published by the given publisher. This is for users to see which publishers are posting more authentic articles in comparison to others. For this reason, each publisher card contains an aggregate score of all the articles from that publisher.

2.4.3 Searching/Filtering Database

The database allows users to search all articles that have been previously authenticated by other users. By using the search bar, users can search for things like article title, author, or publisher. The website also has multiple filter options, some that will filter the report cards themselves and some that filter the publisher cards. The default filters sort by publisher scores from highest to lowest. The two publisher filters are publisher score and publisher name alphabetically. The other filter types filter all the articles, removing the publisher cards they belong to. These include score, title, article date, report creation date, author, number of upvotes and number of downvotes. All of these filters can be sorted in descending or ascending order. The user can also sort by specific publishers and authors. Finally, the user can filter by a minimum article score and number of upvotes and

downvotes. The filter page allows users to apply any filters they want but they can also reset to the default whenever.

2.5 Chrome Extension

The NoCap chrome extension allows users to create an authenticity report for an article the user is currently on. While a user is on an article they wish to authenticate, they can click on the chrome extension. This will start the generation of an authenticity report just like submitting an article URL into the website. Once generated, a popup will appear showing the article title, author, publisher, publication date, score, and summary. It also includes a link to view the full report on the NoCap website Report Page and saving the report into the database.

3. Developer Manual

The developer manual will give current and future developers of NoCap an overview of the system architecture and implementation, including both front end and back end components. The frontend section will go over the different pages and components of the NoCap website and how they interact with one another, delivering information to the user interface to show users after creating an article authenticity report. The backend section will go over the database, AWS Amplify, AWS Nova Lite model, API, and AI prompt used to create the reports from a URL, showing how all the pieces connect to one another to perform the tasks of the product like report creation, storing, and sharing throughout users via the database.

3.1 Frontend

The frontend section describes all the features of the user interface that provides the user with all report and database information. This section is split up into pages and components. The pages consist of the App, which is the main central point for state management and data fetching. The Layout, which is responsible for managing the UI layout across all pages. The final three pages are for each of the main screens of the product; Home, Report, and Database Page. The components are all frontend aspects used on the page, including the sidebar, input bar, search bar, filter menu, report and publisher cards.

3.1.1 Pages

The pages are the specific parts of the UI that let the user perform different tasks depending on the current page they are on. Some pages like the App and Layout are more central and send information throughout the other screen pages to keep formats and viewability consistent. The screen pages are specifically designed for each screen's functionality, displaying specific components and information.

3.1.1.1 App

The **App.jsx** file is the main component of the frontend and central point for state management, data fetching, and app configuration. It initializes the AWS Amplify to interact with the Reports model, which retrieves all report data, storing it in a shared **ReportsContext**. Used to export three React Contexts; **ReportsContext** for managing the list of reports, **SelectedReportContext** for tracking the currently selected report to keep its information on the Report Screen after switching, and **ThemeContext** for light and dark mode toggling, which stays using **localStorage** and falls back to the **prefers-color-scheme** setting when no preference is saved. **updateReportVotes** is used to update up and downvotes for a report. **useEffect** hooks are used to handle initial data loading and themes and their preferences. Finally, it redirects routing responsibilities to **AppRoutes**, separating navigation from state and data concerns.

3.1.1.2 Layout

The **Layout.jsx** file is used for the shared UI structuring of the main page content and the sidebar as it should remain constant on all screens. Outlet is used to render the active page and manages local state to control the sidebar's visibility and opening/closing features displaying backdrop when open for easy navigation.

3.1.1.3 HomePage

The **HomePage.jsx** file is where the **InputBar** and a few sample **ReportCards** are present. Imports the **ReportContext** to access all stored reports and **SelectedReportContext** to set the current report when the user interacts with a report card or generates a new report. Uses **useNavigate** to set the selected report upon clicking a report card and **useMemo** to pick the example cards with the highest, lowest, and median report scores, ensuring no duplicates. The **InputBar** allows the take in of URL for generating a report and creating a report card to be stored in the database while directing the user to the **ReportPage** for viewing.

3.1.1.4 ReportPage

The **ReportPage.jsx** file is responsible for displaying all the authenticity report information including article metadata, report score, summary, detailed analysis, analysis charts, and the up and downvotes. It resolves the target report using URL parameters, navigation state, and global context, ensuring the same output whether the page is accessed from URL submission, report card selection or the extension. It uses **ReportContext** to ensure the UI

is updated upon up or down voting a report, storing the user's votes in **localStorage** and sending the update to **updateReportVotes**. It supports the analysis charts that get the information from the **SelectedReportContext** and gets the claims and descriptors for the given article.

3.1.1.5 DatabasePage

The **DatabasePage.jsx** file is used for searching, storing, and interacting with all stored reports in the system. It consumes global state from **ReportsContext** and **SelectedReportContext** to access and update report data. It also incorporates many components including the **SearchBar**, **FilterMenu**, **ReportCard**, and **PublisherCard**, allowing for searching and multiple filters options, including full sort, author, publisher, and minimum vote and score search. With these filters, there is a publisher view when sorting publisher based filters that will store the articles in the publisher cards and calculate aggregate publisher score. The page supports pagination for both standard article view of 20, 50, and 100, as well as publisher view with a standard of 10 publishers per page, allowing a user to view more of a specific publisher's reports, with a standard of 3 in basic view. The page also allows for up and down voting, updating the global state and backend using **updateReportVotes**.

3.1.2 Components

3.1.2.1 Sidebar

The **Sidebar.jsx** file is the main navigation tool, providing access to core routes and global controls within a consistent UI element. The Sidebar is collapsible whose open or closed state is sent from Layout using **isOpen** and **setOpen** allowing for this via the hamburger button, allowing responsive behavior across device sizes. Navigation between pages uses **NavItem** built into React's **NavLink**, which applies the pages style. It also imports **SelectedReportContext** to navigate to the report page when a report is selected and **ThemeContext** to allow for the light and dark mode button to switch between the two. Finally, Sidebar contains icons for simple understanding of pages while closed and has page names while the Sidebar is open and a footer that contains the NoCap logo including the name when open.

3.1.2.2 InputBar

The **InputBar.jsx** file is used to take user input by accepting an article URL, validates its format, and creates or retrieves the

corresponding report. Manages local state for the input, loading status, and error handling and imports **ReportsContext** and **SelectedReportContext** to update global state. The **InputBar** has a two-step verification, first checking for existing reports using URL comparison. If no report is found, it sends a POST request to the backend API endpoint to analyze the article. The report is either retrieved or added to the global report list and is set to the current report, taking the user to the **ReportPage** for viewing. The **InputBar** supports text area resizing based on URL length, submission via Enter key for accessibility purposes, and popup error messages upon report creation or retrieval failure.

3.1.2.3 SearchBar

The **SearchBar.jsx** file searches through the database with optional filter controls. It maintains local state initialized from **defaultValue** and updates the change using **handleChange**, which invokes **onSearch**. This delegates all search logic allowing real-time filtering without coupling to any specific data source. The filter button is up when **showFiltersButton** is true, allowing **onOpenFilters** to open the popup to allow for more advanced filtering.

3.1.2.4 FilterMenu

The **FilterMenu.jsx** file implements the filter options popup, defaulting the filters using **initialFilters**. It maintains state values initialized by **buildInitialValues**, combining default filter options with any **appliedValues**, ensuring filters remain when the popup is closed and reopened. The popup is managed through local state **isOpen** with external control via **forwardRef** and **useImperativeHandle**, allowing the filter button to open the menu. Users can update the filters via checkboxes, radio, and range values, and **handleApply** validates the current filters via **onApply**, which then closes the menu. Finally, it features locking scrolling while the menu is open as well as closing the menu using the Escape key, ensuring proper behavior while open.

3.1.2.5 ReportCard

The **ReportCard.jsx** file displays summarized report data, including article metadata, authenticity score, and up and down vote controls. The card maintains local state **displayUp**, **displayDown**, and **localVote**, supporting immediate feedback upon submitting a vote. The votes are grabbed from **localStorage** so the user interactions remain between sessions. **handleVote**

works by adjusting local counts before evoking **onVote**. Specific utility functions are used such as **ensureProtocol** to normalize incoming data and **getDisplayURL and formatDate** to ensure proper link and date formats to display on the cards.

3.1.2.6 PublisherCard

The **PublisherCard.jsx** file groups reports by the same publisher and displays their publisher name, aggregate score, and a list of **ReportCards** inside. It uses **expanded** and **hiddenCount** to support a collapsible layout, displaying the **ReportCards** not in view of the **COLLAPSED_MAX_HEIGHT** using **ResizeObserver** and a **checkOverflow** function. This allows for a "Show '#' more" or "Show less" to appear whether the **PublisherCard** is expanded or collapsed.

3.1.2.7 AnalysisCharts

The **AnalysisCharts.jsx** file creates two visualizations using the **recharts** library to summarize report analytics. It constructs two datasets, **claimsData** and **descriptorsData**, representing factual claim accuracy and descriptor sentiment. They both use the **BarChart** one vertical and one horizontal using **layout="vertical"**. Each chart is wrapped in a **ResponsiveContainer** so it scales across different screen sizes.

3.1.3 AppRoutes

The **AppRoutes.jsx** file is the main routing structure using **react-router-dom**, organized in a nested layout with routes to the **HomePage** using **index**, **ReportPage** using **/report** and **/report/:id**, and **DatabasePage** using **/database** layered into the **Layout** route. **/report/:id** allows the **ReportPage** to render specific reports based on the report ID. It wraps the application in a BrowserRouter using the Layout to ensure consistent UI elements like the **Sidebar** and the header to remain between page routes.

3.2 Backend

The backend is responsible for article content retrieval, AI-powered factuality analysis, data persistence, and authentication. It is split across two layers: a locally hosted Python service that handles article fetching and AI communication, and a cloud-based AWS Amplify infrastructure that manages the database, file storage, and user authentication.

3.2.1 Database

The database is powered by AWS DynamoDB, accessed through AWS AppSync via a GraphQL API. The Reports model stores all article

authenticity report data, including the article title, author, publisher, publication date, URL, report creation date, authenticity score, summary, and detailed analysis. The database is configured with public API Key authorization, allowing any user to read, create, and delete reports. The Amplify-generated client on the frontend uses this GraphQL API to list, create, and delete report entries directly, keeping the database in sync with the global ReportsContext state.

3.2.2 AWS Amplify

AWS Amplify serves as the cloud infrastructure layer for the NoCap application. It provisions and manages three core AWS services: Amplify Auth using Amazon Cognito for email-based user authentication, Amplify Data using AWS AppSync and DynamoDB for the GraphQL-backed Reports database, and Amplify Storage using Amazon S3 for public file storage under the **reportsStorage** bucket. The frontend initializes Amplify using the generated **amplify_outputs.json** configuration file, which provides all endpoint, API key, and region details needed to communicate with these services.

3.2.3 AWS AI Nova

The AI analysis is performed using **Amazon Nova Lite v1 (us.amazon.nova-lite-v1:0)** via the AWS Bedrock Runtime service, deployed in the **us-east-1** region. When an article URL is submitted, the backend extracts the article text and sends it to Nova Lite along with a structured prompt. The model returns a JSON response containing an authenticity score from 0 to 100, a 2–3 sentence summary explaining the score, a 4–6 sentence detailed analysis citing specific claims, a list of factual claims classified as correct or incorrect, and a list of descriptors used before people and entities labeled by sentiment. AWS credentials are sourced from the local environment using the boto3 SDK.

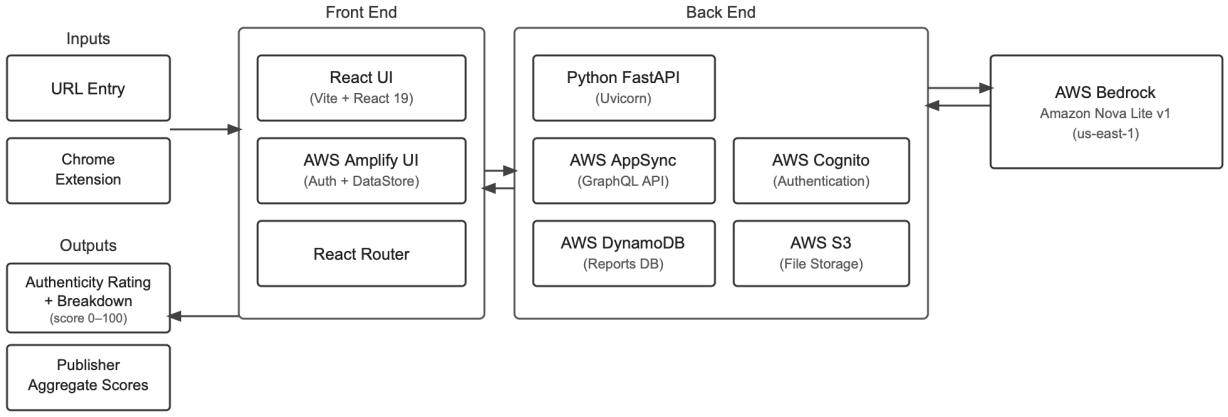
3.2.4 API

The API is a Python FastAPI application served by Uvicorn, running locally on **http://localhost:8000**. It exposes a single primary endpoint, **POST /api/analyze**, which accepts a JSON body containing an article URL. CORS middleware is configured to allow requests from any origin, enabling both the React frontend and the Chrome extension to communicate with it. Upon receiving a request, the endpoint calls the **analyze_url** function from **analyze_article.py**, which orchestrates content extraction and AI analysis. The endpoint returns a JSON response containing the success status and the full analysis result. A **GET /health** endpoint is also available for health checks.

3.2.5 AI Prompt

The AI prompt is defined in **ai_prompt.py** and is responsible for constructing the request sent to Amazon Nova Lite and parsing its response. The **grade_article_factuality** function receives the extracted article text and sends it to Bedrock with a structured system prompt instructing the model to act as a fact-checking assistant. The prompt specifies the exact JSON schema the model must return, including the score, summary, detailed analysis, claims array, and descriptors array. The function then parses the raw model output, extracts the JSON block, and returns a structured Python dictionary. If parsing fails, an error is raised and propagated back to the API endpoint.

3.3 System Architecture



System Architecture Diagram